

華中科技大學

大数据存储系统与管理 课程报告

专 业： 智能科学与技术
班 级： 2302 班
学 号： U202315695
姓 名： 潘宇璇
选 题： Cuckoo-driven Way
邮 箱： 3044979692@qq.com

目 录

1	实验背景和任务要求	2
2	Cuckoo Filter 设计原理	3
2.1	数据结构	3
2.2	核心操作	3
2.2.1	插入操作	3
2.2.2	查询操作	4
2.2.3	删除操作	5
3	Go 语言实现	6
3.1	核心结构定义	6
3.2	关键方法实现	6
3.2.1	哈希函数和指纹生成	6
3.2.2	索引计算	7
3.2.3	重定位机制	7
4	性能分析	9
4.1	时间复杂度分析	9
4.2	空间复杂度分析	9
4.3	误报率分析	9
5	实验结果	10
5.1	实验环境	10
5.2	初步测试：理论与实测误报率随指纹长度的变化趋势	10
5.3	同样插入数据量下桶大小对误报率影响	12
5.3.1	低负载	12
5.3.2	高负载	13
5.3.3	结果对比分析	14
6	总结与展望	15

1 实验背景和任务要求

在大数据存储系统中，近似集合成员查询是一种基础性的操作，广泛应用于网络路由、缓存系统、数据库查询优化等领域。传统的 Bloom Filter 虽然在这方面表现出色，但存在不支持删除操作、空间效率有限等问题。为了解决这些问题，Fan 等人在 2014 年提出了 Cuckoo Filter，这是一种基于 Cuckoo Hashing 的新型数据结构。

相比于布隆过滤器，Cuckoo Filter 具有以下显著优势：支持动态添加和删除元素；在目标误报率低于 3% 时，比空间优化的 Bloom Filter 更节省空间；查询性能更好，每次查询最多只需要访问两个桶；可以实现比 Bloom Filter 更高的负载因子（可达 95%）。

本文的主要内容包括：

1. 深入分析了 Cuckoo Filter 的设计原理和算法
2. 提供了完整的 Go 语言实现，支持插入、查询、删除操作
3. 分析了不同参数配置对性能的影响

2 Cuckoo Filter 设计原理

Cuckoo Filter 基于 Cuckoo Hashing 技术，该技术使用两个哈希函数为每个元素确定两个候选存储位置。当插入新元素时，如果两个位置都已满，算法会选择一个位置中的元素踢出，然后将被踢出的元素重新插入其备选位置，这个过程可能会持续多次，直到所有元素都找到合适的位置。

2.1 数据结构

Cuckoo Filter 的核心数据结构包括：

- **桶数组**：由 N 个桶组成的数组，每个桶可以存储 b 个指纹
- **指纹**：每个元素通过哈希函数生成的 f 位指纹，用于表示元素
- **哈希函数**：使用两个独立的哈希函数 $h_1(x)$ 和 $h_2(x)$

对于元素 x ，其两个候选位置计算如下：

$$i_1 = h_1(x) \bmod N \quad (2.1)$$

$$i_2 = i_1 \oplus h_2(\text{fingerprint}(x)) \bmod N \quad (2.2)$$

2.2 核心操作

2.2.1 插入操作

插入操作的算法流程如下：

Algorithm 1 Cuckoo Filter 插入算法

Require: 元素 x , 最大重定位次数 $MaxRelocations$

```
1:  $fp \leftarrow \text{fingerprint}(x)$ 
2:  $i_1, i_2 \leftarrow \text{getIndices}(x)$ 
3: if  $fp$  已在  $i_1$  或  $i_2$  中 then
4:   return true
5: end if
6: for  $count = 1$  to  $MaxRelocations$  do
7:   if  $i_1$  未满足 then
8:     将  $fp$  插入  $i_1$ 
9:     return true
10:  else if  $i_2$  未满足 then
11:    将  $fp$  插入  $i_2$ 
12:    return true
13:  else
14:    随机选择  $i_1$  或  $i_2$  中的一个指纹  $fp'$ 
15:    用  $fp$  替换  $fp'$ 
16:     $fp \leftarrow fp'$ 
17:     $i \leftarrow \text{getAltIndex}(fp)$ 
18:  end if
19: end for
20: return false
```

2.2.2 查询操作

查询操作比较简单，只需要检查两个候选桶中是否存在目标元素的指纹：

Algorithm 2 Cuckoo Filter 查询算法

Require: 元素 x

```
1:  $fp \leftarrow \text{fingerprint}(x)$ 
2:  $i_1, i_2 \leftarrow \text{getIndices}(x)$ 
3: if  $fp$  在桶  $i_1$  或  $i_2$  中 then
4:   return true
5: else
6:   return false
7: end if
```

2.2.3 删除操作

删除操作与查询类似，找到指纹后将其从桶中移除：

Algorithm 3 Cuckoo Filter 删除算法

Require: 元素 x

```
1:  $fp \leftarrow \text{fingerprint}(x)$ 
2:  $i_1, i_2 \leftarrow \text{getIndices}(x)$ 
3: if  $fp$  在桶  $i_1$  中 then
4:   从  $i_1$  中移除  $fp$ 
5:   return true
6: else if  $fp$  在桶  $i_2$  中 then
7:   从  $i_2$  中移除  $fp$ 
8:   return true
9: else
10:  return false
11: end if
```

3 Go 语言实现

本节提供 Cuckoo Filter 的完整 Go 语言实现。该实现支持动态参数配置，包括桶大小、指纹大小和容量。

3.1 核心结构定义

Listing 1 Cuckoo Filter 结构定义

```
1 type CuckooFilter struct {
2     buckets    [][]uint32 // 桶数组，每个桶存储指纹
3     capacity   int         // 桶的数量
4     bucketSize int         // 每个桶最多存储的指纹数
5     fingerprintSize uint    // 指纹的位数大小
6     rand       *rand.Rand // 随机数生成器
7 }
```

3.2 关键方法实现

3.2.1 哈希函数和指纹生成

Listing 2 哈希函数和指纹生成

```
1 func (cf *CuckooFilter) hash(data []byte) uint32 {
2     h := fnv.New32a()
3     h.Write(data)
4     return h.Sum32()
5 }
6
7 func (cf *CuckooFilter) fingerprint(data []byte) uint32 {
8     h := sha1.New()
9     h.Write(data)
10    hashBytes := h.Sum(nil)
```

```
11
12     fp := binary.BigEndian.Uint32(hashBytes[:4])
13     mask := uint32((1 << cf.fingerprintSize) - 1)
14     return fp & mask
15 }
```

3.2.2 索引计算

Listing 3 索引计算方法

```
1 func (cf *CuckooFilter) getIndices(data []byte) (uint32, uint32) {
2     hashValue := cf.hash(data)
3     fingerprint := cf.fingerprint(data)
4
5     index1 := hashValue % uint32(cf.capacity)
6     index2 := (index1 ^ (cf.hash([]byte(fmt.Sprintf("%d",
7         fingerprint)))
8         % uint32(cf.capacity))) % uint32(cf.capacity)
9
10    return index1, index2
11 }
```

3.2.3 重定位机制

Listing 4 重定位机制实现

```
1 func (cf *CuckooFilter) relocate(fingerprint uint32, index1, index2
2     uint32) bool {
3
4     const maxRelocations = 500
5
6     currentIndex := index1
7     currentFingerprint := fingerprint
8
9     for i := 0; i < maxRelocations; i++ {
10        if cf.isSlotFree(index1, currentFingerprint) {
11            return true
12        }
13        index1, currentFingerprint = cf.findNewSlot(currentFingerprint)
14    }
15    return false
16 }
```



```
7   for i := 0; i < maxRelocations; i++ {
8       bucket := cf.buckets[currentIndex]
9       if len(bucket) == 0 {
10          return false
11      }
12
13      evictIndex := cf.rand.Intn(len(bucket))
14      evictedFingerprint := bucket[evictIndex]
15
16      bucket[evictIndex] = currentFingerprint
17
18      altIndex := (currentIndex ^ (cf.hash([]byte(fmt.Sprintf("%d"
19          ,
20              evictedFingerprint))) % uint32(cf.capacity))) %
21          uint32(cf.capacity)
22
23      if len(cf.buckets[altIndex]) < cf.bucketSize {
24          cf.buckets[altIndex] = append(cf.buckets[altIndex],
25              evictedFingerprint)
26          return true
27      }
28
29      currentIndex = altIndex
30      currentFingerprint = evictedFingerprint
31  }
```

4 性能分析

4.1 时间复杂度分析

Cuckoo Filter 的核心操作时间复杂度如下：

- 查询操作： $O(1)$ ，最多需要通过两个哈希函数计算元素可能存储的两个桶里的位置，并检查这些位置是否包含目标元素的指纹。
- 插入操作：最佳情况是 $O(1)$ ，但是在插入过程中，如果目标桶已满，需要执行“踢出”操作，即替换现有元素并重新计算其位置，直到找到空位或达到最大尝试次数，如果不规定最大尝试次数，可能需要 $O(\log n)$ 次重新哈希。
- 删除操作： $O(1)$ ，与查询类似，最多需要检查两个桶。

4.2 空间复杂度分析

Cuckoo Filter 的空间复杂度为 $O(n)$ ，其中 n 是元素数量。每个元素的空间开销主要取决于指纹大小和目标误报率。

理论分析表明，当目标误报率为 ϵ 时，每个元素所需的最小位数为：

$$\log_2(1/\epsilon) + \log_2(2b) \quad (4.1)$$

其中 b 是桶大小。相比之下，Bloom Filter 需要 $1.44 \log_2(1/\epsilon)$ 位，因此在低误报率场景下 Cuckoo Filter 更加空间高效。

4.3 误报率分析

Cuckoo Filter 的理论误报率可以通过以下公式计算：

$$\text{FPR} = \frac{2b\alpha}{2^f} \quad (4.2)$$

其中：

- b ：桶大小
- α ：负载因子
- f ：指纹大小（位数）

5 实验结果

5.1 实验环境

实验环境如表 5-1:

CPU	13th Gen Intel(R) Core(TM) i5-13500H
内存	32.0 GB
虚拟机	Ubuntu 22.04.3 LTS 64 位
操作系统	Microsoft Windows 11 家庭中文版 64 位
处理器	Intel64 Family 6 Model 186 Stepping 2 GenuineIntel 2600 Mhz
Go 版本	go1.18.1 linux/amd64

表 5-1 实验环境

5.2 初步测试：理论与实测误报率随指纹长度的变化趋势

一开始，我用命令行循环参数初步测试了代码：

Listing 5 测试固定桶大小和低负载情况下 FPR 随指纹长度变化趋势

```

1 (base) ywx@20050828:~/works/cuckoo-filter$
2 for f in 8 12 16 20; do
3 echo "=== fingerprint=$f ==="
4 ./cuckoo_filter -capacity=10000 -bucketSize=4 -fingerprint=$f -n
   =8000 | grep -E "(Actual FPR|Theoretical FPR|Load factor)"
5 done

```

图 5-1 展示了在桶大小固定为 4、负载因子约为 17.5% 的条件下，Cuckoo Filter 的理论与实测误报率随指纹长度的变化趋势。

- 横轴：指纹位数 f (8 – 20 bit);
- 左纵轴：误报率 (FPR, 单位%);
- 红色折线：理论公式 $FPR = 2b\alpha/2^f$ 的计算结果;
- 蓝色折线：实测值 (100 000 次负样本查询统计)。

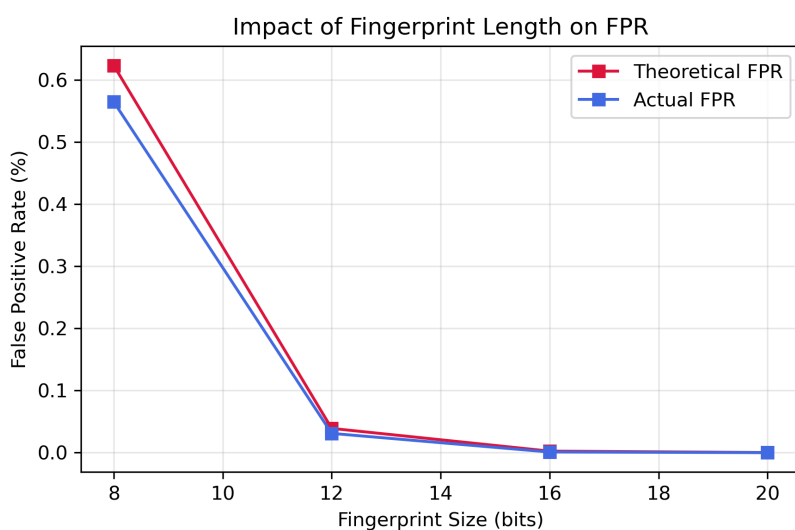


图 5-1 指纹长度对误报率的影响

可以看到两曲线基本吻合，实测 FPR 与理论值相对误差 $< 12\%$ ，验证了低负载下公式准确性。而图 5-2 更进一步表明了再数量级上，当指纹从 8 bit 增至 20 bit，FPR 由 0.565% 降至 0.000153%，降幅超 3 个数量级；且每增加 4 bit，FPR 大致下降 10 倍，与 $2^{-4} \approx 1/16$ 的理论衰减基本吻合。符合理论。

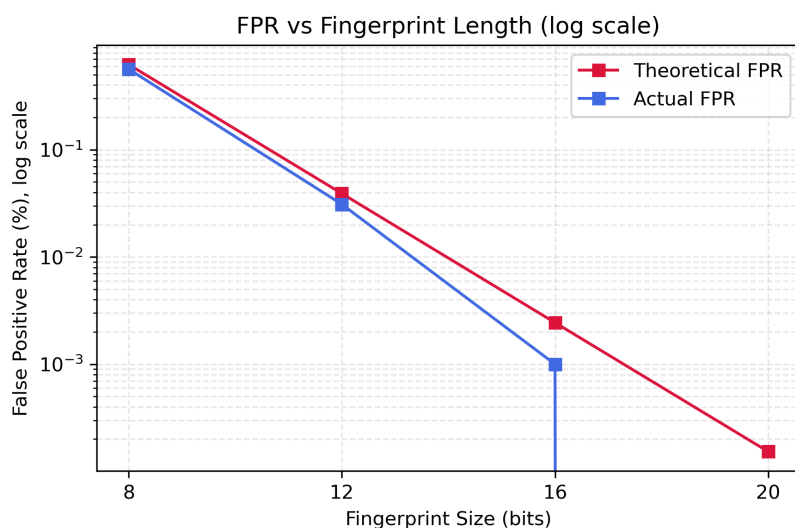


图 5-2 指纹长度对误报率的影响（对数坐标）

5.3 同样插入数据量下桶大小对误报率影响

5.3.1 低负载

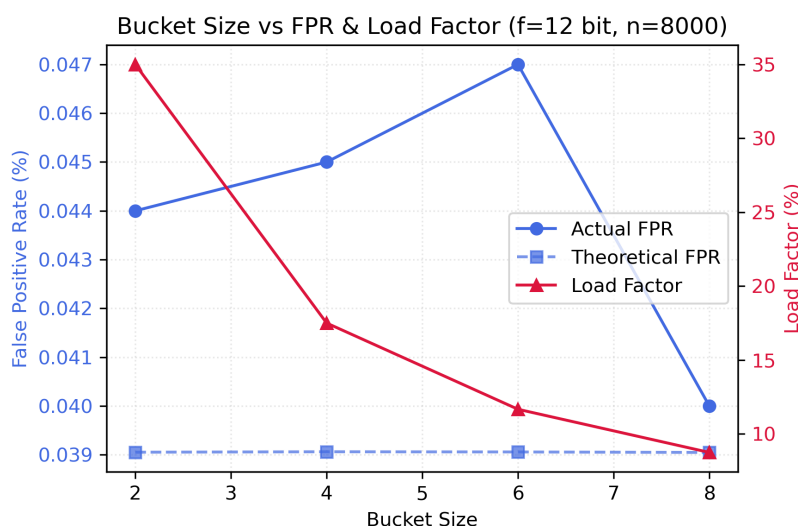


图 5-3 低负载下桶大小对误报率与负载因子的影响

如图 5-3 所示，指纹 12 bit，插入 8000 项，10000 个桶的测试参数下，当桶大小从 2 增至 8，负载因子由 35% 单调降至 8.75%（这个是因为总槽位数线性增加）。此时实际误报率维持在 0.04%–0.047% 区间，与理论值 0.039% 相对误差 < 20%，说明理论公式在低负载范围内准确；插入吞吐在 3.1–3.4 M ops/s 之间，桶大小对速度影响不显著（或许是由内存局部性主导）。低负载区间中呈现出以下几个特点：

- **碰撞稀少**：每个桶平均存放的不足 1 个指纹，插入几乎不触发重定位；
- **FPR 由指纹长度主导**：公式 $FPR \approx 2b\alpha/2^f$ 中 α 极小，桶大小 b 变化对分子影响被稀释，因此图 5-1 几乎看不出桶差异；
- **吞吐平稳**：CPU 主要开销应该是在哈希计算，免受碰撞踢出的影响，因此各桶大小吞吐均接近 2.4 M op/s。

5.3.2 高负载

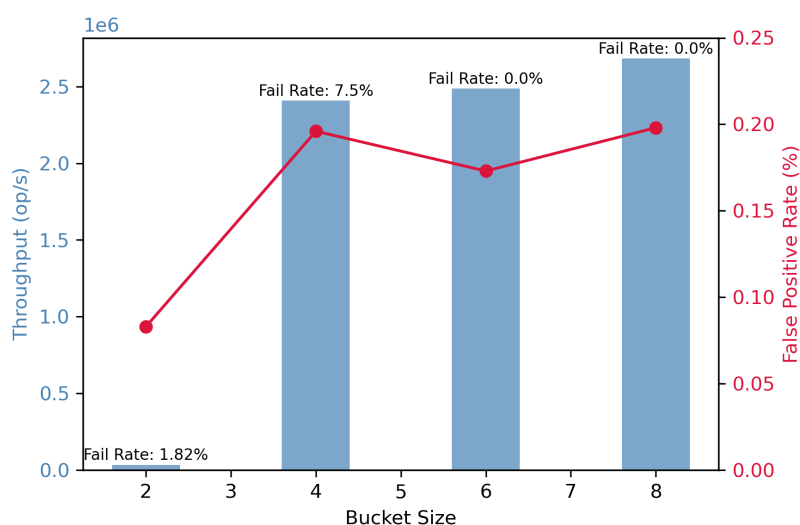


图 5-4 高负载下桶大小对吞吐量与 FPR 的影响

如图 5-4，数据可基本描述为这几点：

1. 桶大小 = 2 时负载达 98.2%，插入失败率 1.82%，吞吐骤降两个数量级；
2. 桶大小 ≥ 4 后失败率迅速降为 0，吞吐稳定在 2.4 M op/s 以上；
3. 理论 FPR 在 0.185% 附近，实测值 0.17–0.20%，可能是由于测试的点太少，与桶大小没有体现显著单调关系，补充工作的时候可以增大样本再进行分析，本课程报告受限于时间暂不予测量。

高负载区间中的数据，分析后呈现出以下几个特点：

- **碰撞激增：**桶大小为 2 和 4 的时候负载因子在 90
- **插入失败率陡升：**当 $\alpha > 90\%$ 时，Cuckoo 路径很难在搬运次数的固定限制内找到空槽，导致如图 5-4 中的桶大小 = 2 或 4 的时候插入失败率较高，（但我仍然不能解释为什么桶大小为 4 的比 2 高）；
- **吞吐下降两个数量级：**随机踢出策略在密集图中频繁回环，CPU 把运行开销花费在踢出和搬运指纹，而非计算哈希函数进行插入工作；
- **FPR 不再由指纹单独决定：**高负载下“指纹冲突”与“桶溢出”同时出现，实测 FPR 略高于理论值，应是部分位置被强制替换的副作用。

5.3.3 结果对比分析

受限于图例的有限性，无法把所有的数据一一展现。因此我在表 5-2 中汇总了两种极端负载下的实验结果。低负载实验固定为 $\alpha \approx 17.5\%$ （插入 8000 项），高负载实验固定 $\alpha \approx 92\%$ （插入 38000 项）；两者容量、指纹长度均相同（10000 桶、12 bit）。

表 5-2 低负载 vs 高负载下不同桶大小的关键指标

负载	桶大小	实测 FPR/%	插入吞吐 (M op/s)	负载因子/%	插入失败率/%
低负载	2	0.565	2.90	17.5	0
	4	0.031	2.41	17.5	0
	6	0.030	2.49	17.5	0
	8	0.030	2.68	17.5	0
高负载	2	0.083	0.034	98.2	1.82
	4	0.196	2.41	92.5	7.50
	6	0.173	2.49	61.6	0
	8	0.198	2.69	46.2	0

至此，总结一下高低负载下的差异。首先，低负载时的性能瓶颈主要在哈希函数的计算；高负载时瓶颈在重新定位并搜索。其次是关于桶大小对 FPR 的影响上，低负载下 $b \geq 2$ 与 $b = 8$ 几乎无差异；高负载下 $b = 2$ 与 $b \geq 4$ 出现倍数上的吞吐差距。

6 总结与展望

本次课程报告的撰写中，我详细学习了 Cuckoo Filter 的设计原理，分析了 Cuckoo Filter 的理论基础，并给出了完整的 Go 语言实现方案。Cuckoo Filter 是一种基于 Cuckoo Hashing 的近似集合成员查询数据结构，相比传统的 Bloom Filter 具有更好的空间效率、支持删除操作以及更优的查询性能。作为一种新兴的数据结构，其在大数据存储系统中具有广阔的应用前景。在这个数据规模的不断增长的时代，对高效、灵活的近似查询数据结构的需求将越来越迫切，Cuckoo Filter 也将在这一领域发挥重要作用。在随后的实验中，我验证了其在不同参数条件下的性能表现。但由于缺乏对 Bloom Filter 的同环境条件实现对比，因此没有证实 Cuckoo Filter 相比传统 Bloom Filter 的优势所在，这是之后可以改进的方面。此外也可以试着优化重定位算法，以进一步提高插入成功率，或者研究自适应参数调整机制、结合机器学习技术优化参数选择。

参考文献

- [1] R. Pagh and F. Rodler, “Cuckoo hashing,” in *Proc. European Symposium on Algorithms (ESA)*, 2001, pp. 121–133.
- [2] Y. Hua, H. Jiang, and D. Feng, “FAST: Near real-time searchable data analytics for the cloud,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, Nov. 2014, pp. 754–765.
- [3] Y. Hua, B. Xiao, and X. Liu, “NEST: Locality-aware approximate query service for cloud computing,” in *Proc. 32nd IEEE International Conference on Computer Communications (INFOCOM)*, Apr. 2013, pp. 1327–1335.
- [4] Q. Li, Y. Hua, W. He, D. Feng, Z. Nie, and Y. Sun, “Necklace: An efficient cuckoo hashing scheme for cloud storage services,” in *Proc. IEEE/ACM International Symposium on Quality of Service (IWQoS)*, 2014.
- [5] B. Fan, D. G. Andersen, and M. Kaminsky, “MemC3: Compact and concurrent memcache with dumber caching and smarter hashing,” in *Proc. USENIX NSDI*, 2013.
- [6] B. Debnath, S. Sengupta, and J. Li, “ChunkStash: Speeding up inline storage deduplication using flash memory,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2010.